BÀI TẬP THỰC HÀNH ĐỌC, GHI FILE

Nguyễn Văn Bảo – 25732641

1. Demonstration 3…

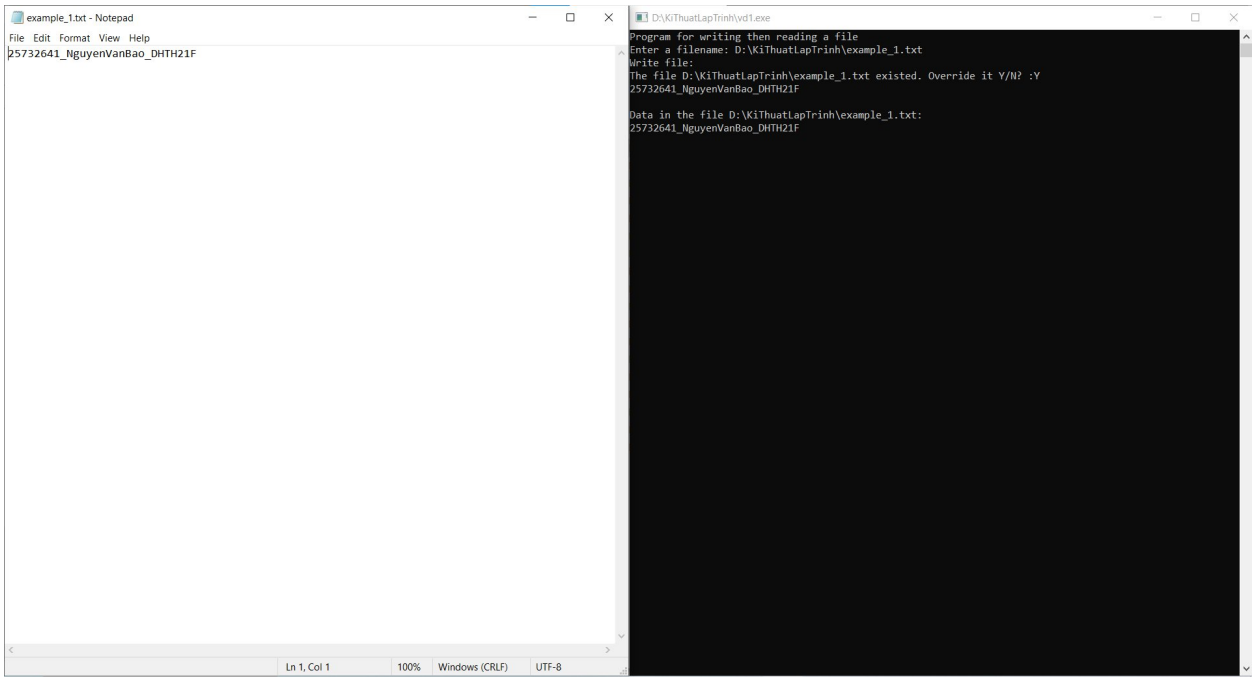
a. Code

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
int exist(char* filename) {
    FILE* f = fopen(filename, "r");
    if (f != NULL) {
        fclose(f);
        return 1;
    }
    return 0;
}
int writeFile(char* filename) {
    if (exist(filename) == 1) {
        printf("The file %s existed. Override it Y/N? :", filename);
        if (toupper(getchar()) == 'N') return 0;
    }
    char line[201];
    int length = 0;
    FILE* f = fopen(filename, "w");
    fflush(stdin);
    do {
        gets(line);
        length = strlen(line);
        if (length > 0) fprintf(f, "%s\n", line);
    } while (length > 0);
    fclose(f);
    return 1;
}
```

```
int printFile(char* filename) {
    if (exist(filename) == 0) {
        printf("The file %s does not exist.\n", filename);
        return 0;
    }
    FILE* f = fopen(filename, "r");
    char line[201];
    while (fscanf(f, "%[^\n]%*c", line) > 0) puts(line);
    fclose(f);
    return 1;
}
int main() {
    char filename[81];
    printf("Program for writing then reading a file\n");
    printf("Enter a filename: ");
    gets(filename);

    printf("Write file:\n");
    if (writeFile(filename) == 1) {
        printf("Data in the file %s:\n", filename);
        if (printFile(filename) == 0) printf("File error!\n");
    } else {
        printf("Writing file fail!\n", filename);
    }
    getchar();
    return 0;
}
```

b. Kết quả:



c. Giải thích

- **Chức năng chính của chương trình:**
  - Kiểm tra sự tồn tại của tệp
  - Ghi dữ liệu vào tệp, cho phép người dùng nhập nhiều dòng văn bản từ bàn phím và lưu chúng vào tệp. Có cơ chế hỏi xác nhận nếu muốn ghi đè lên tệp cũ (Y/N).
  - Đọc dữ liệu từ tệp, từ đó truy xuất dữ liệu để hiển thị lên màn hình console
- **Vai trò của các hàm trong chương trình:**
  - int exist(): mở tệp ở chế độ đọc ("r"). Nếu mở được (khác NULL) thì trả về 1 (tồn tại), ngược lại trả về 0.
  - int writeFile(): Thực hiện việc tạo hoặc mở tệp để ghi ("w"). Nó chứa vòng lặp do...while để lấy dữ liệu từ hàm gets() cho đến khi người dùng nhập một dòng trống.
  - int printfFile(): Mở tệp để đọc. Sử dụng định dạng fscanf đọc từng dòng văn bản (bao gồm cả khoảng trắng) và in ra màn hình bằng puts.
  - int main(): Lấy tên tệp từ người dùng, sau đó gọi lần lượt hàm ghi và hàm đọc.
- **Luồng thực hiện của chương trình:**

- Chương trình ban đầu yêu cầu người dùng nhập tên tệp qua biến filename
- Gọi hàm **writeFile**: Trước khi ghi, hàm exist được gọi ngầm để kiểm tra tệp. Nếu tệp đã có, người dùng phải chọn **'Y'** để tiếp tục. Người dùng nhập nội dung. Khi một dòng trống (chỉ nhấn Enter) được nhập, tệp sẽ được đóng lại (fclose).
- Nếu writeFile thành công (trả về 1), chương trình tiếp tục gọi printFile. Nếu thất bại hoặc người dùng từ chối ghi đè, chương trình thông báo "Writing file fail!".
- Gọi hàm **printFile**: Kiểm tra tệp có tồn tại không. Nếu có, mở tệp và quét từng dòng để hiển thị kết quả "Data in the file...".
- Dừng màn hình bằng getchar(), người dùng xem kết quả trước khi đóng chương trình.

2. Demonstration 4

a. Code

```
#include <stdio.h>

void fileToArray(char* fname, int* a, int* pn) {
    FILE* f = fopen(fname, "r");
    if (f == NULL) return;
    fscanf(f, "%d", pn);
    int i;
    for (i = 0; i < *pn; i++) {
        fscanf(f, "%d", &a[i]);
    }
    fclose(f);
}

void asc_sort(int *a, int n) {
    int i, j, t;
    for (i = 0; i < n - 1; i++) {
        for (j = n - 1; j > i; j--) {
            if (a[j] < a[j - 1]) {
```

```
                t = a[j];
                a[j] = a[j - 1];
                a[j - 1] = t;
            }
        }
    }
}

void print(int *a, int n) {
    int i;
    for (i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
}
void printToFile(char* fname, int* a, int n) {
    FILE* f = fopen(fname, "w");
    fprintf(f, "%d ", n);
    int i;
    for (i = 0; i < n; i++) {
        fprintf(f, "%d ", a[i]);
    }
    fclose(f);
}

int main() {
    char infName[] = "array1.txt";
    char outfName[] = "array2.txt";
    int a[200];
    int n = 0;

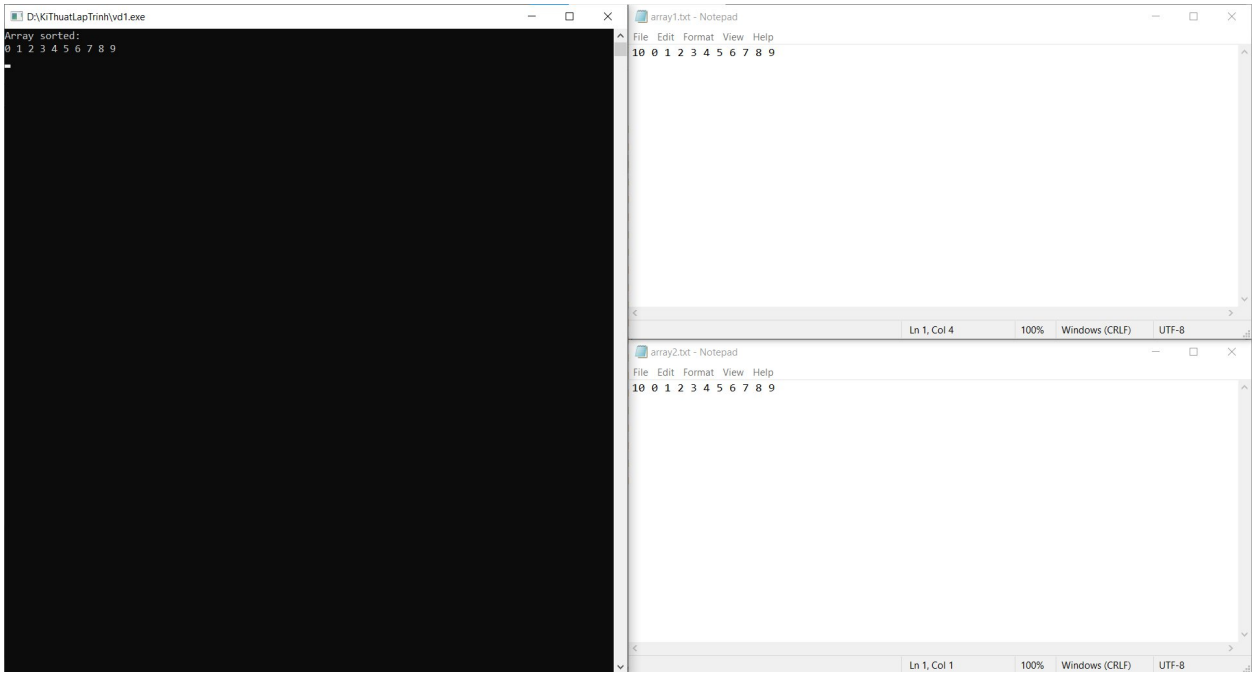
    fileToArray(infName, a, &n);
    asc_sort(a, n);

    printf("Array sorted:\n");
    print(a, n);
```

```
    printToFile(outfName, a, n);

    getchar();
    return 0;
}
```

b. Kết quả:



c. Giải thích:

- **Các chức năng chính của chương trình:**
  - Đọc dữ liệu số (Input): Lấy danh sách số nguyên từ một tệp tin văn bản có sẵn (array1.txt).
  - Xử lý thuật toán :Thực hiện sắp xếp các số nguyên này theo thứ tự tăng dần.
  - Xuất dữ liệu (Output): Hiển thị kết quả ra màn hình Console và lưu lại danh sách đã sắp xếp vào một tệp tin mới (array2.txt).
- **Vai trò của các hàm trong chương trình:**
  - **void fileToArray():** Mở tệp nguồn để đọc dữ liệu. Dòng đầu tiên trong file được dùng để xác định số lượng phần tử (pn), sau đó dùng vòng lặp để đưa các số tiếp theo vào mảng a.

- **void asc_sort():** Sử dụng thuật toán sắp xếp nổi bọt (Bubble Sort) để so sánh các cặp số liền kề và hoán đổi chúng nếu số đứng trước lớn hơn số đứng sau.
- **void print(...):** Duyệt qua mảng đã xử lý và in từng phần tử ra màn hình để người dùng quan sát kết quả tức thời.
- **int printToFile(...):** Mở tệp đích ở chế độ ghi mới ("w"). Ghi số lượng phần tử và dãy số đã sắp xếp vào tệp để lưu trữ lâu dài.
- **int main():** Khởi tạo các biến cần thiết (mảng a, biến n), thiết lập tên tệp và gọi các hàm con theo đúng trình tự logic.

- **Luồng thực hiện của chương trình:**
  - Hàm main xác định tệp đầu vào là array1.txt.
  - Gọi hàm fileToArray. Tại đây, con trỏ tệp sẽ quét qua tệp tin để chuyển dữ liệu từ ổ cứng vào bộ nhớ RAM (mảng a). Sau khi hoàn tất, tệp được đóng lại ngay để giải phóng bộ nhớ.
  - Hàm asc_sort thực hiện sắp xếp trên mảng a.
  - Hàm print hiển thị dãy số lên màn hình. Hàm printToFile tạo ra tệp array2.txt và "đổ" dữ liệu từ mảng vào tệp này.
  - Người dùng nhấn phím bất kỳ (getchar) trước khi đóng hoàn toàn.

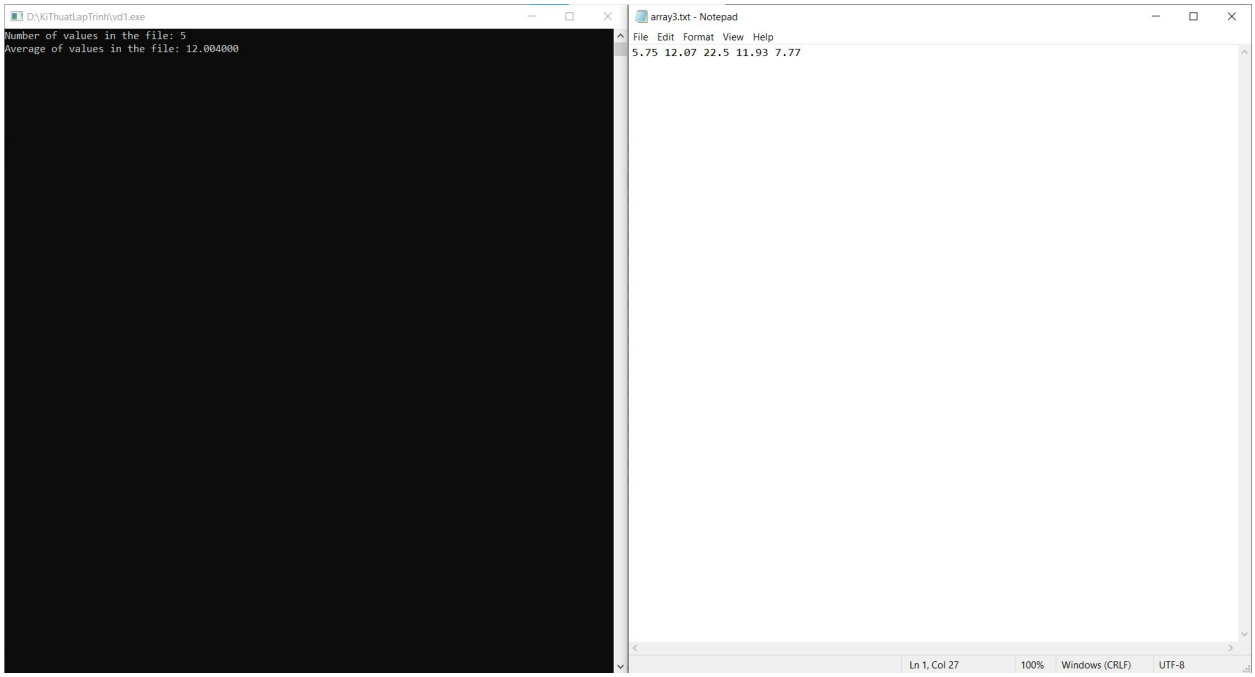
3. **Demonstration 5**
   a. **Code**

```
#include <stdio.h>

void processFile(char* fname, int* pcount, double* psum) {
    FILE* f = fopen(fname, "r");
    if (f == NULL) return;
    *pcount = 0;
    *psum = 0;
    double x;
    while (fscanf(f, "%lf", &x) == 1) {
        (*pcount)++;
        (*psum) += x;
```

```
    }
    fclose(f);
}
int main() {
    char infName[] = "array3.txt";
    int count = 0;
    double sum = 0;
    processFile(infName, &count, &sum);
    printf("Number of values in the file: %d\n", count);
    if (count > 0) {
        printf("Average of values in the file: %lf\n", sum / count);
    } else {
        printf("File is empty or not found.\n");
    }
    getchar();
    return 0;
}
```

b. Kết quả:



c. Giải thích:

- **Chức năng chính của chương trình:**

- Tự động duyệt và lấy toàn bộ các số thực có trong tệp array3.txt.
- Đếm tổng số lượng các con số hiện có và tính tổng giá trị của chúng. Sau đó, tính toán giá trị trung bình cộng.
- Hiển thị số lượng các số đã đọc và giá trị trung bình cộng ra màn hình console.

- **Vai trò của các hàm trong chương trình:**
  - void processFile(): Mở tệp ở chế độ đọc ("r"). Sử dụng vòng lặp để duyệt qua từng số thực cho đến khi hết tệp. Biến con trỏ pcount và psum được dùng để cập nhật dữ liệu trực tiếp về hàm main.
  - Khởi tạo các biến lưu trữ (count, sum), thiết lập tên tệp nguồn. Sau khi gọi hàm xử lý, nó thực hiện phép tính chia (sum / count) để tìm trung bình cộng và in kết quả.
- **Luồng thực hiện của các hàm:**
  - Trong hàm main, các biến count và sum được gán bằng 0 để chuẩn bị được gán dữ liệu.
  - Hàm processFile được gọi để đọc tệp array3.txt. Mỗi khi fscanf tìm thấy một số thực, nó sẽ tăng count lên 1 và cộng dồn giá trị đó vào sum. Vòng lặp while (fscanf(...) == 1) cho phép chương trình tự dừng lại khi gặp cuối tệp (EOF) hoặc gặp ký tự không phải là số, giúp xử lý được các file có độ dài bất kỳ.
  - Sau khi đọc tệp được đóng lại ngay sau khi đọc xong để đảm bảo an toàn dữ liệu.
  - Hàm main nhận lại các giá trị đã cập nhật. Nó kiểm tra nếu count > 0 (để tránh lỗi chia cho 0) thì sẽ tính trung bình cộng và in ra màn hình với định dạng số thực (%lf).